

Modül 17: Standart Kütüphane Turu

Sürekli kullanacağınız dört köşe, her birinin insanlara gerçek zaman kaybettiren bir tuzağı olduğu için seçildi.

Buradaki her örnek **tekrarlanabilir**: sabit saatler, sabit tarihler, tohumlanmış rastgele üreticiler. Bunun bir sebebi dersin kararlı olması, bir sebebi de zamanı ve rastgeleliği tekrarlanabilir kılmamanın test yazmak için ihtiyacınız olan tekniğin kendisi olması.

Neler Öğreneceksiniz

- Ezberlemeye değer String metotları ve `formatted`
- `java.time` : hangi tipi seçmeli, ve aritmetiği neden sınırlıyor
- Tarih işleminin gerçekte bozulduğu yer olan yaz saati uygulaması
- Gerçek hatalara yol açmış beş `Math` davranışı
- Tekrarlanabilir rastgele sayılar, ve onları asla ne zaman kullanmamalı

Başka Bir Dilden Geliyorsanız

	Python	Java
Biçimlendirme	f-string'ler	<code>"%s".formatted(x)</code>
Tarih	<code>datetime</code>	<code>LocalDate</code> , <code>LocalDateTime</code> , <code>ZonedDateTime</code>
Saat dilimi farkındalığı	<code>tzinfo</code> ile isteğe bağlı	seçtiğiniz tiple isteğe bağlı
Süre	<code>timedelta</code>	<code>Duration</code> ve <code>Period</code> , ve farklılar
Rastgele	<code>random.seed(42)</code>	<code>RandomGeneratorFactory.of(...).create(42)</code>

Önemli olan satır: Java size dilim taşıyan ya da taşımayan bir tip seçtirir. `LocalDateTime` bir zaman anı **değildir**, ve onu öyle saymak en yaygın `java.time` hatasıdır.

Ders

String'ler

`Strings.java` dosyasından:

```
" x ".strip() // Unicode-aware; trim() is not
"a,b,c".split(",") // takes a REGEX, not a literal
String.join("-", "2026", "09")
"ab".repeat(3)
"%-10s|%8.2f|%,d".formatted("name", 3.14159, 1_234_567)
```

iki tuzak:

- split** **düzenli ifade alır.** Kaçış olmadan `.` ya da `|` üzerinde bölmek yanlış cevap verir, ve bu klasik bir bir saatlik hatadır.
- compareTo** **negatif bir sayı, sıfır ya da pozitif bir sayı döndürür.** `-1`, `0`, `1` değil. Aksini varsayan kod yanlıştır.

Ezberlemeye değer biçimlendirme dönüşümleri: `%s` her şey, `%d` tam sayı, `%.2f` iki ondalık basamak, `%n` platform satır sonu, `%,d` binlik ayracı, `%%` gerçek yüzde işareti. `%10s` sağa, `%-10s` sola yaslar.

java.time: tipi seçmek

Tip	Tuttuğu	Kullanım
<code>LocalDate</code>	yalnızca tarih	doğum günü
<code>LocalTime</code>	yalnızca saat	açılış saatleri
<code>LocalDateTime</code>	ikisi, dilim yok	duvar saati okuması
<code>ZonedDateTime</code>	bir yerdeki gerçek an	toplantı
<code>Instant</code>	UTC zaman çizgisinde bir nokta	zaman damgası
<code>Duration</code>	saat, saniye	geçen süre
<code>Period</code>	yıl, ay, gün	takvim miktarları

`LocalDateTime` dilim taşımaz. `2026-09-23T10:15` belirtilmemiş bir yerdeki bir andır. Birini saklayıp UTC varsaymak, bu modülün en çok kaçınmanızı istediği hatadır.

Her `now()` isteğe bağlı bir `Clock` alır. Argümansız `now()` çağıran kod, "29 Şubat'ta ne olur" sorusu için test edilemez. Bir `Clock` bağımlılık olarak alın ve sorun kaybolur.

Sınırlayan aritmetik

`DateAndTime.java` dosyasından:

```
31 Jan + 1 month: 2026-02-28
29 Feb + 1 year: 2025-02-28
and back again: 2026-01-28
so +1 month then -1 month is NOT the identity.
```

İkisi de ileri sarmak yerine son geçerli güne sınırlar. Bu bilinçlidir ve **geri alınabilir değildir**. Faturalama kodunuz bir ay ekleyip bir ay çıkarıyorsa, başladığı yere inmez.

`Duration` zaman tabanlı ve tamdır. `Period` takvim farkındalıktır, dolayısıyla bir ay uygulandığı yere göre 28, 29, 30 ya da 31 gündür. İkisinin de var olma sebebi budur.

`ofPattern` metoduna her zaman bir `Locale` verin. Onsuz makinenin varsayılanını kullanır, dolayısıyla aynı kod yurt dışındaki bir sunucuda farklı biçimlendirir. Java 18 öncesindeki varsayılan karakter kümesiyle aynı hata sınıfı.

Yaz saati

`TimeZonesAndDst.java` dosyasından:

```
--- the spring forward gap ---
01:30 EST:      2026-03-08T01:30-05:00[America/New_York]
plus one hour:  2026-03-08T03:30-04:00[America/New_York]
02:30 does not exist; atZone gives: 2026-03-08T03:30-04:00[America/New_York]

--- the autumn fall-back overlap ---
first  (EDT): 2026-11-01T01:30-04:00[America/New_York]
second (EST): 2026-11-01T01:30-05:00[America/New_York]
```

Orada üç şey oluyor:

- 01:30 üzerine bir saat eklemek **03:30** veriyor, çünkü 02:00 hiç var olmadı. Aritmetik doğru; bir gerçek saat geçti.
- Boşluktaki yerel bir saat hiç yoktur. Java fırlatmaz, ileri taşır.
- Sonbaharda 01:30 **iki kez** yaşanır. `atZone` erken ofseti seçer. Geç olanı istiyorsanız söylemeniz gerekir, ve sessizce birini varsaymak rezervasyonların iki kez sayılma biçimidir.

Ve en çok önem taşıyan ayrım:

`plusDays(1)` duvar saatini korur. `plusHours(24)` 24 gerçek saat ekler. Yılda iki gün farklı cevaplar verirler. "Yarın aynı saatte" ile "şu andan yirmi dört saat sonra" farklı işlemlerdir.

Sizi güvende tutan kurallar: bir `Instant` ya da UTC saklayın ve yalnızca gösterim için dönüştürün, sabit ofset yerine `Europe/London` gibi bölge `ZoneId` kullanın, ve gerçek bir an için asla `LocalDateTime` saklamayın.

Math

`MathSurprises.java` dosyasından:

```
Math.abs(MIN_VALUE):    -2147483648
is it negative?         true
abs(MIN_VALUE) % 7      = -2    <- a negative array index
floorMod(MIN_VALUE, 7) = 5     <- always in range

-7 / 2                  = -3
Math.floorDiv           = -4

Math.round(2.5)         = 3
Math.round(-2.5)        = -2

MAX_VALUE + 1           = -2147483648
Math.addExact throws: integer overflow
```

abs negatif bir sayı döndürebilir. `int` aralığı asimetriktir, dolayısıyla en küçüğü negatiflemek kendisine sarar. Bu önemlidir çünkü `Math.abs(hash) % buckets` yaygın bir deyimdir ve yukarıdaki çıktının gösterdiği gibi **negatif indeks** üretir. Onun yerine her zaman aralıkta olan `Math.floorMod(hash, buckets)` ya da fırlatan `Math.absExact` kullanın.

Örneğin 8 değil 7 kova kullandığına dikkat edin. `MIN_VALUE` sayısı 8'e tam bölünür, dolayısıyla 8 ile her iki biçim de `0` döndürür ve hata gizli kalır. Başarısızlığın gerçekten görüldüğü bir örnek seçmek için bir parçası.

Tam sayı bölmesi sıfıra doğru keser; `floorDiv` negatif sonsuza doğru yuvarlar. Negatiflerde farklılaşırlar.

round yarımı pozitif sonsuza doğru yuvarlar, dolayısıyla `-2.5` sonucu `-3` değil `-2` olur.

Taşma sessizdir. `addExact`, `subtractExact` ve `multiplyExact` sarmayı reddeder. Sarmalanmış bir değerin çökmeden daha kötü olduğu her yerde kullanın, ki para ya da bellek sayan her şey için bu her zaman geçerlidir.

Rastgele sayılar

`RandomNumbers.java` dosyasından:

```
RandomGenerator rng = RandomGeneratorFactory.of("L64X128MixRandom").create(42);
```

Biçime dikkat edin: üretici **fabrika** oluşturur. `RandomGenerator.of(name)` doğrudan bir üreteç döndürür ve `create` metodu yoktur.

Aynı tohum, aynı dizi. Karıştıran ya da örnekleyen bir testin sabit bir tohuma ihtiyacı vardır, aksi hâlde ayda bir kimsenin tekrar üretmediği sebeplerle başarısız olur. Üretici, bir `Clock` alır gibi parametre olarak alın.

`nextInt(bound)` sınırı dışarıda bırakır; `nextInt(low, high)` alt sınırı içerir, üst sınırı dışarıda bırakır.

Bunları güvenlikle ilgili hiçbir şey için kullanmayın. Sözde rastgeledirler ve yeterli çıktı görüldüğünde tahmin edilebilirler. Token, oturum kimliği ve anahtar için `java.security.SecureRandom` kullanın.

Bir üretici iş parçacıkları arasında paylaşmayın. `java.util.Random` senkronizedir ve darboğaz olur; `ThreadLocalRandom.current()` her iş parçacığına kendininkini verir.

Çalıştırın

```
java modules/17-standard-library-tour/examples/Strings.java
java modules/17-standard-library-tour/examples/DateAndTime.java
java modules/17-standard-library-tour/examples/TimeZonesAndDst.java
java modules/17-standard-library-tour/examples/MathSurprises.java
java modules/17-standard-library-tour/examples/RandomNumbers.java

./scripts/verify-examples.sh modules/17-standard-library-tour
```

Herhangi birini iki kez çalıştırın, çıktı birebir aynıdır, çünkü hiçbirisi geçerli zamana ya da tohumuz bir üretece bağlı değildir.

Sık Yapılan Hatalar

Gerçek bir an için `LocalDateTime` saklamak. Dilimi yoktur. Bir `Instant` saklayın.

Bölge kimliği yerine sabit ofset kullanmak. `+01:00` Londra'da yılın yarısı yanlışır.

`Clock` olmadan `now()` çağırmak. Kod tarihe bağlı her dal için test edilemez hâle gelir.

`Locale` olmadan `ofPattern`. Yurt dışındaki bir sunucuda farklı çıktı.

Kova indeksi için `Math.abs(hash) % n`. Dört milyarda bir negatif. `floorMod` kullanın.

`double` değerlerini `==` ile karşılaştırmak. Bir tolerans içinde karşılaştırın, ya da para için kayan noktadan kaçın.

Testte tohumuz bir üretici. Ayda bir, kimsenin tekrar üretmediği bir kararsızlık.

`split(".")`. O, herhangi bir karakteri eşleyen bir düzenli ifadedir.

Özet Çıkarımlar

- `java.time` tipini dilim gerekip gerekmediğine göre seçin. `LocalDateTime` dilim taşımaz.
- `Clock` ve `RandomGenerator` değerlerini bağımlılık olarak alın, böylece zaman ve rastgelelik test edilebilir olur.

- Tarih aritmetiği sınırlar ve geri alınabilir değildir.
- `plusDays(1)` ile `plusHours(24)` yaz saati günlerinde farklıdır.
- `abs` negatif olabilir, bölme keser, `round` asimetriktir, taşma sessizdir. `Exact` metotları bunun yerine fırlatır.
- Testler için tohumlayın; önemli olan her şey için `SecureRandom` kullanın.

Ödev

[homework/README.md](#)

Yaz saatinden sağ çıkan bir planlama aracı kurun, sonra dört `Math` tuzağını yeniden üretin. Referans çözüm [solutions/17-standard-library-tour/](#) altında.